

Groma, explained from the ground up

A complete walkthrough of the drone photogrammetry and camera coverage planning system, assuming no prior knowledge of photogrammetry, computer vision, geodesy, or camera optics.

This is the companion to `groma-design.md`, which is the compressed engineering version. Nothing here is left out of that one; this one just explains why each piece exists and how it works.

Contents

1. [What the system is for](#)
 2. [Turning photographs into a 3D model](#)
 3. [Where things actually are: coordinates and heights](#)
 4. [From a mess of points to a set of objects](#)
 5. [Measuring things](#)
 6. [Cameras and what they can see](#)
 7. [Occlusion: what is hidden behind what](#)
 8. [Tents, scenarios, and choosing where cameras go](#)
 9. [How the software is put together](#)
 10. [How you know any of it is correct](#)
 11. [Build order](#)
 12. [Everything that will go wrong](#)
 13. [Glossary](#)
-

1. What the system is for

You fly a drone over a sports ground. You want to end up with a tool where you can click on

a light mast, say "put a 4-megapixel camera here, pointing that way, with this lens", and immediately see a coloured map of the ground showing exactly which parts of the site that camera can usefully see — and how that changes when you erect twelve event tents in the middle of the pitch.

There are five capabilities stacked on top of each other. Each is only as good as the one beneath it.

- 5. Coverage recomputed with temporary structures in the way
 - └ 4. Camera placement, field of view, image quality on target
 - └ 3. Measuring distances, heights and areas
 - └ 2. Structures identified and approved by a person
 - └ 1. A metrically correct 3D model of the site
 - └ good drone capture

Reading upward is the natural way to build it. Reading downward tells you where the system's credibility comes from: a beautiful coverage map computed on a badly scaled model is a confident-looking wrong answer.

The one design decision that shapes everything

Here is the decision worth understanding before anything else, because the rest of the system is arranged around it.

Coverage is never computed against the raw 3D model. It is computed against a small set of simple shapes that a human has approved.

A photogrammetric model of a football ground is roughly 20–80 million triangles of noisy geometry. Fences come out lumpy. Light masts come out half-missing, because a 20 cm pole photographed from 60 metres up is only a few pixels wide. There are floating blobs where a car drove through during the survey. There are trees, which will look completely different in six months.

If you ray-trace camera sightlines through that, you get shadows that aren't real, gaps that aren't real, a computation that takes minutes instead of milliseconds, and a result nobody can audit — because "why is this corner dark?" has no answer except "some triangles are there".

So the system converts that mess into a **proxy model**: a few hundred clean primitives — cylinders for masts, thin boxes for fence runs, boxes for buildings, plus a terrain height grid. A person reviews each one and accepts, rejects, or reclassifies it.

That review step is the requirement in the brief about "user's selection or rejection of such structures for accuracy". It is not a cosmetic feature. It is the mechanism that turns unusable

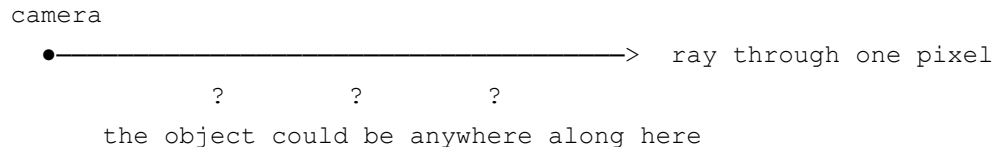
geometry into a computable model. And it means every dark patch on the final coverage map traces back to a named object that somebody signed off.

The proxy model is the product. The photogrammetry is just a cheap way to obtain it.

2. Turning photographs into a 3D model

2.1 Why this is hard at all

A camera takes a 3D world and squashes it onto a 2D sensor. Every pixel in a photograph corresponds not to a point in space but to a **ray** — a line running out from the camera through that pixel into the world. Something along that ray produced the pixel, but the photograph cannot tell you how far away it was.

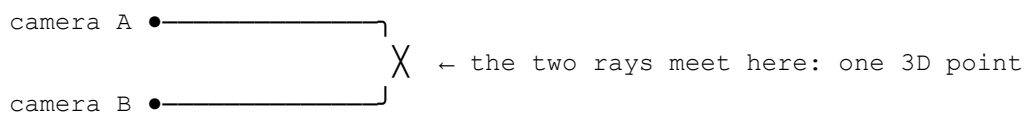


One photo, therefore, cannot give you 3D. Two photos of the same object from different positions can.

2.2 Parallax and triangulation

Hold your thumb up and close one eye, then the other. Your thumb appears to jump against the background. That apparent shift is **parallax**, and it depends on how far away the thumb is — near things shift a lot, far things shift a little.

Two cameras looking at the same point in the world each produce a ray. If you know where both cameras were and which way they were facing, the two rays intersect at exactly one point in space. That's **triangulation**, and it's the whole basis of photogrammetry.



Do this for millions of points visible in many overlapping photographs and you get a 3D model.

The catch is circular: to triangulate you need to know the camera positions, and the only way to work out the camera positions is from the photographs themselves. That circularity is what Structure from Motion solves.

2.3 Finding the same point in two photographs

Before anything else, you need to know that pixel (1420, 883) in photo 17 and pixel (902, 1150) in photo 18 are looking at the same physical thing.

This is done with **feature detection and matching**:

1. **Detect features.** An algorithm scans each image for small patches that are distinctive and would be recognisable from a different angle — corners, blobs, texture junctions. A flat patch of grass is useless (every patch looks the same); a corner of a penalty box marking is excellent. Classic algorithms: SIFT, AKAZE, SuperPoint.
2. **Describe them.** Each feature gets a **descriptor** — a vector of maybe 128 numbers summarising the local pattern of gradients around it, constructed so it stays roughly the same if the patch is seen from a different distance, rotation, or brightness.
3. **Match them.** For each feature in photo A, find the feature in photo B with the most similar descriptor. Most of these matches are right; some are wrong.
4. **Filter geometrically.** If two images are of the same rigid scene, all correct matches must obey a geometric constraint (the *epipolar constraint*: a point in image A must lie on a particular line in image B, determined by the relative pose of the two cameras). Matches that violate it are discarded. This is usually done with RANSAC, explained in §4.4.

This is why **texture matters enormously**. A freshly resurfaced tennis court that is uniformly green will reconstruct badly, because there is nothing distinctive to match. Grass, gravel, painted lines and worn tarmac all reconstruct well.

2.4 Structure from Motion (SfM)

SfM resolves the circularity by bootstrapping.

1. Pick two photos with lots of matched features and a good baseline (taken from reasonably different positions). From the matches alone, you can recover their *relative* pose — where camera B is relative to camera A — up to an unknown scale.
2. Triangulate the matched features to get an initial cloud of 3D points.
3. Take a third photo. Some of its features match points you've already triangulated. Given 2D-to-3D correspondences you can solve for that camera's pose directly (this is the *Perspective-n-Point* problem).
4. Triangulate its new features. Add them to the cloud.
5. Repeat for every photo.

6. Periodically, run **bundle adjustment** over everything.

The result is a **sparse point cloud** (typically 10^5 – 10^6 points) plus, crucially, the **pose of every camera** — where each photograph was taken from and which way it was pointing.

Those camera poses are usually treated as a by-product and thrown away. This system keeps them, because §4.5 needs them.

2.5 Camera intrinsics, extrinsics, and reprojection error

Three terms you'll meet constantly.

Extrinsics — where the camera was and how it was oriented. Six numbers: three for position, three for rotation.

Intrinsics — the internal geometry of the camera itself. Focal length in pixels, the position of the optical centre on the sensor, and **lens distortion** coefficients. Real lenses aren't perfect pinholes; wide lenses bow straight lines outward (barrel distortion). Photogrammetry either uses factory calibration or, more usually, solves for the distortion parameters as part of the optimisation — *self-calibration*.

Reprojection error — the fundamental quality measure. Take a reconstructed 3D point. Using the solved camera pose and intrinsics, mathematically project it back onto the image. Compare where it lands to where the feature was actually detected. The distance between them, in pixels, is the reprojection error.

detected feature •
 \
reprojected point ○ ← this gap, in pixels, is the reprojection error

A good aerial reconstruction has a root-mean-square reprojection error of 0.5–1.5 pixels across millions of observations. Above about 2 px, something is wrong.

Bundle adjustment is the process of nudging all camera poses, all intrinsics, and all 3D point positions simultaneously to minimise the total squared reprojection error. It's a very large non-linear least-squares problem — for a 1,000-image survey, hundreds of thousands of unknowns — solved with Levenberg–Marquardt exploiting the fact that most cameras don't see most points, so the problem matrix is mostly zeros.

The name comes from the *bundle* of light rays converging on each camera centre; the optimisation adjusts the bundles until they agree.

2.6 Sparse to dense: Multi-View Stereo

SfM gives you camera poses and a sparse cloud. It doesn't give you a surface — a million

points over a football ground is a point every few decimetres.

Multi-View Stereo (MVS) takes the now-known camera poses and computes depth for *every pixel* of every image, not just at features. Because the poses are known, the search is one-dimensional: for a given pixel in image A, the corresponding pixel in image B must lie somewhere along a known line, and you slide along it looking for the best photometric match.

Merging all those per-pixel depths gives a **dense point cloud**: tens to hundreds of millions of points, spaced at roughly the ground sample distance.

2.7 The standard output products

From the dense cloud, a few derived products:

Product	What it is	Format
Dense point cloud	Every reconstructed point, with colour	LAS / LAZ
Mesh	The points connected into a triangle surface	OBJ / PLY / glTF
Textured mesh	The mesh with the original photos projected onto it	OBJ + texture atlas
DSM — Digital Surface Model	A height grid of the <i>top</i> of everything: buildings, trees, fences	GeoTIFF
DTM — Digital Terrain Model	A height grid of the bare ground, with everything above it removed	GeoTIFF
Orthophoto	A photograph reprojected so it has no perspective — every pixel viewed straight down, so you can measure directly off it	GeoTIFF (COG)

The DSM/DTM distinction matters here. A DSM includes a tent's roof; a DTM is the ground the tent stands on. Coverage evaluation needs the DTM (people walk on the ground) and the occlusion model needs whatever is between the DTM and the DSM.

An orthophoto is worth understanding because it's counterintuitive. A normal aerial photo

has perspective: buildings near the edge of the frame lean outward, and you can see their sides. In an orthophoto every pixel is as if photographed from directly above it, so a rectangle on the ground is a rectangle in the image and distances scale uniformly. It is essentially a map made of photographs.

2.8 The scale problem, which is the single biggest trap

Everything in §2.4 was recovered from image geometry alone. Image geometry cannot determine size.

A photograph of a real football ground and a photograph of a perfect 1:100 scale model of that ground, taken from proportionally scaled positions, are **pixel-for-pixel identical**. There is no information in the images to distinguish them.

So a raw SfM reconstruction is correct in *shape* and arbitrary in *size*. It's determined only up to a **similarity transform**: seven degrees of freedom — three of position, three of rotation, one of scale.

This is dangerous specifically because it is invisible. The model looks perfect. You measure the pitch and get 94.2 metres. It is wrong by a constant factor and nothing in the model tells you so.

Three ways to fix it, best first:

1. RTK or PPK on the drone. *Real-Time Kinematic* GNSS. An ordinary GPS receiver is accurate to a few metres. RTK adds a second receiver at a known fixed point (a base station, or a network of them). Because both receivers see the same satellites through the same atmosphere, the errors are nearly identical and mostly cancel — so the *relative* position between them can be resolved to 1–2 cm. The drone therefore knows where each photograph was taken to within a couple of centimetres, and feeding those positions into the bundle adjustment fixes all seven degrees of freedom at once. *PPK* (Post-Processed Kinematic) does the same arithmetic afterwards from logged data instead of over a live radio link, which is more robust.

2. Ground control points (GCPs). Physical targets — high-contrast chequerboards — placed on the site and surveyed to centimetre accuracy with a GNSS rover or total station. You mark each target in several photographs, and the bundle adjustment ties the model to those known coordinates. Slower, still the accuracy benchmark. Use at least 5, spread across the site and, importantly, varied in height — GCPs all at the same elevation constrain scale horizontally but leave the vertical weakly determined.

3. Scale bars only. One or more objects of precisely known length lying in the scene. Gives you correct dimensions but no absolute position or orientation.

Always keep check points. These are additional surveyed points that are deliberately

excluded from the bundle adjustment. After solving, you compare the model's coordinates for those points against their surveyed truth. The residuals are the only honest statement of accuracy the system can make. Points used in the fit always look good — that's what fitting means.

A free check specific to your domain: sports pitches have regulated dimensions. A full-size football pitch is 105 × 68 m; a FIBA basketball court 28 × 15 m; a tennis court 23.77 × 10.97 m. Measure the reconstructed markings and compare. It costs nothing, needs no survey equipment, and catches a scale error immediately. Build it in as an automatic post-reconstruction check.

2.9 Ground sample distance: your accuracy ceiling

GSD is how many metres of ground one image pixel covers. It is the resolution limit of everything downstream.

$$\text{GSD (metres per pixel)} = \frac{\text{sensor_width_mm} \times \text{altitude_m}}{\text{focal_length_mm} \times \text{image_width_px}}$$

For a DJI Mavic 3E (17.3 mm sensor width, 12.3 mm lens, 5280 px wide):

Altitude	GSD
40 m	1.07 cm/px
60 m	1.60 cm/px
100 m	2.67 cm/px

Rules of thumb: horizontal accuracy lands around 1–3 × GSD, vertical around 2–4 × GSD. So if the requirement is dimensioning to ±2 cm, you must fly at roughly 35–40 m and accept the extra flight time and battery swaps. Flying at 100 m and promising 2 cm is not a matter of better software.

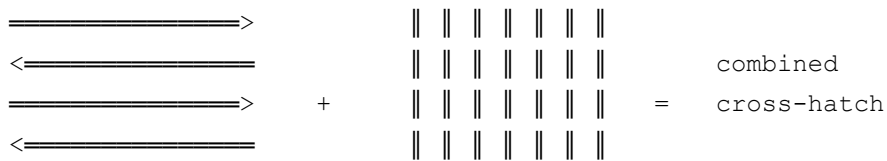
2.10 Flight planning

Overlap. Every point on the ground needs to appear in many photographs from different angles — more views means better triangulation and more redundancy against bad matches. Standard practice for survey-grade work:

- **Front overlap** (between consecutive photos along a flight line): 80%
- **Side overlap** (between adjacent flight lines): 70%

Yes, that means each point appears in a dozen or more photos. That redundancy is what makes the result robust.

Nadir grid. “Nadir” means straight down. Fly a lawnmower pattern with the camera pointing down. Then fly a second grid perpendicular to the first — a *cross-hatch*. This dramatically improves the geometry and helps the self-calibration.



Oblique orbits. A nadir-only flight reconstructs a flat pitch beautifully and reconstructs a light mast as a smear of noise. Think about why: from directly above, a vertical pole is a small circle. You never see its side, you never get two views of the same point on it from meaningfully different directions, and its silhouette is a few pixels of dark against grass. So add:

- A **perimeter orbit** with the gimbal at 45°, one full circuit of the site.
- A **dedicated orbit around every mast you intend to mount a camera on.** You are going to hang hardware off that structure and compute sightlines from its top. Its geometry needs to be right, not approximately right.

Sun and wind. Overcast is ideal — hard shadows move between photos taken minutes apart, which confuses feature matching, and dark shadows contain no texture. Avoid flying near solar noon in summer for the same reason. Wind causes motion blur and gimbal drift.

2.11 Video versus stills

Video is tempting. One continuous pass, no intervalometer, no worrying about trigger spacing. It is also substantially worse, for three reasons:

Rolling shutter. Most drone sensors don’t expose the whole frame at once; they read it out row by row over a few milliseconds. If the camera is moving during readout, the top of the frame is exposed from a slightly different position than the bottom, and the image is geometrically skewed. Photogrammetry assumes a single consistent projection per image, so this injects error directly into the poses.

Inter-frame compression. Video codecs store a few full frames (I-frames) and describe the rest as changes from their neighbours. The reconstruction of a non-key frame is smoothed and approximate. Feature detectors depend on exactly the high-frequency detail that compression throws away first.

Motion blur. Video frame rates force short exposures that are still long relative to the

camera's motion, and there's no opportunity to slow down for each shot.

Expect 2–4× worse reprojection error from video than from stills of the same scene.

Support video anyway, because sites get surveyed by whoever is available with whatever they have — but gate it:

- Extract frames at intervals driven by the desired overlap, not at a fixed rate.
- Prefer I-frames.
- Score every candidate frame for sharpness (variance of the Laplacian — a measure of how much high-frequency detail is present) and reject blurry ones.
- **Show the operator the rejection rate at ingest time.** If 60% of frames were rejected, they need to know before the model is built, not after.

2.12 Which reconstruction software

Option	Verdict
OpenDroneMap (ODM)	Primary. Purpose-built for drone survey. Reads EXIF and RTK metadata, accepts GCP files, and produces the whole product set — orthophoto, DSM, DTM, point cloud, textured mesh. Docker-packaged, permissively licensed, runs headless via NodeODM.
COLMAP	Fallback. The reference implementation for SfM accuracy. Slower and more manual, and it doesn't produce survey products out of the box. Use it when ODM fails to register a difficult set.
GLOMAP	A <i>global</i> SfM approach — solves all camera poses at once rather than adding images one at a time. Dramatically faster on large sets. Worth benchmarking once surveys routinely exceed ~2,000 images.
Pix4D / DJI Terra / RealityCapture	Commercial, excellent, closed, per-seat licensed. The right response is to support importing their outputs. An organisation that already owns Pix4D should not have to re-process.
Gaussian splatting / NeRF	Visualisation only. Never measurement.

Two of those rows need expanding, because ODM and COLMAP are the two you will actually use and they are different kinds of thing.

COLMAP is a reconstruction engine. It does exactly what §2.4 and §2.6 describe and

nothing else: find features, match them, solve camera poses, undistort, dense stereo, fuse. It is the accuracy benchmark others are measured against. You run it as six explicit commands:

```
colmap feature_extractor    # find distinctive patches in every image
colmap sequential_matcher  # match them between image pairs
colmap mapper              # Structure from Motion → poses + sparse cloud
colmap image_undistorter   # remove lens distortion, ready for dense matching
colmap patch_match_stereo  # Multi-View Stereo – per-pixel depth    [needs CUDA]
colmap stereo_fusion       # merge the depth maps into a dense cloud
```

What it gives you: a pose and camera model for every image, a sparse cloud, and a dense cloud. What it does not give you: georeferencing, scale, orthophoto, DSM, DTM, or any GCP workflow. The output sits in an arbitrary coordinate frame at an arbitrary size, and tying it to the real world is a separate manual step (`colmap model_aligner`). Dense stereo requires an NVIDIA GPU; there is no practical CPU path.

`pip install pycolmap` gives you the first three steps from Python. **GLOMAP** is a 2024 drop-in replacement for the `mapper` step that solves all poses at once instead of adding images one at a time — much faster on large sets, comparable accuracy.

OpenDroneMap is a survey pipeline. It is not a reconstruction algorithm; it is an orchestrator that chains about a dozen open-source tools together and adds all the geospatial machinery COLMAP lacks:

```
OpenSfM      → poses + sparse cloud, reading EXIF/RTK, handling GCPs
OpenMVS      → dense cloud and mesh
MVS-Texturing → project the photographs onto the mesh
PDAL / Entwine → point cloud filtering and classification
GDAL         → DSM, DTM and orthophoto as georeferenced GeoTIFFs
```

You run it as one command against a folder containing `images/` and optionally `gcp_list.txt`, and it produces the whole product set from §2.7 in real-world coordinates. That is the difference that matters: ODM's output can be measured and overlaid on a site plan; COLMAP's cannot, until you do the georeferencing yourself.

The `gcp_list.txt` format is worth knowing since you will be generating it. First line is the coordinate system, then one row per observation of a target in an image — easting, northing, height, pixel-x, pixel-y, filename:

```
EPSG:2326
832451.221 816003.874 12.334 2104 1518 DJI_0421.JPG
832451.221 816003.874 12.334 1877 990  DJI_0422.JPG
```

Each target needs marking in at least three images.

NodeODM wraps ODM in a REST API — post a zip of images and options, poll for progress, download the results. That is what the system talks to, rather than the command line, because it gives progress reporting for free.

The trade-off between them: ODM handles everything you need and fails as a black box somewhere in the middle of a two-hour pipeline. COLMAP fails loudly at a specific step and hands you the tools to understand why, but leaves the geospatial half of the problem to you. Hence: ODM primary, COLMAP as the escape hatch for image sets ODM cannot register.

One practical note. ODM ships as a Docker image, and AutoDL instances are themselves containers where Docker-in-Docker is usually unavailable. Check `docker info` on any box before planning around ODM running there; the alternatives are to reconstruct elsewhere and import the artefacts, or to build COLMAP from source on the GPU box.

That last row deserves explanation because it is currently the most fashionable thing in 3D reconstruction and the most likely to be proposed by someone.

3D Gaussian Splatting represents a scene not as a surface but as millions of small translucent, coloured, stretched blobs. Rendering is a matter of splatting them onto the screen in depth order. The results are photorealistic and it's fast. But there is no surface — nothing to snap a measurement to, nothing to ray-trace an occlusion against, and no principled way to say where the fence is as opposed to a fog of blobs that looks like a fence from the training viewpoints. Mesh extraction from splats exists and is improving, but it is not survey-grade.

The right use is as a **walkthrough view** offered alongside the metric model: a photorealistic representation for showing a client what the site looks like. It must never be the thing measurements or coverage are computed on, and the interface should make it obvious which mode you're in.

3. Where things actually are: coordinates and heights

This section looks like housekeeping and is where projects of this type most often die quietly.

3.1 Latitude/longitude is not metres

Latitude and longitude are angles on a curved surface. One degree of latitude is about 111 km everywhere. One degree of longitude is 111 km at the equator and zero at the poles. You cannot do geometry in degrees.

So for anything computational you use a **projected coordinate reference system**: a mathematical flattening of the curved Earth onto a plane, with coordinates in metres. Every projection distorts something — area, angle, or distance — and each is designed to keep distortion small over a specific region.

Relevant ones:

- **EPSG:2326** — Hong Kong 1980 Grid. Local grid, metres, used on HK site plans.
- **EPSG:32650** — UTM Zone 50N. Metres, covers Hong Kong, portable and widely supported.
- **EPSG:4326** — WGS84 latitude/longitude. What GPS gives you. Not metres. Fine for storing a site's rough location; useless for computing on.

"EPSG" is just a registry of numbered coordinate system definitions. Quoting the EPSG code is how you say unambiguously what a coordinate means.

Rule for this system: everything geometric is stored in one projected CRS in metres, recorded per site. Conversions happen at exactly two places — ingest and export — and nowhere else.

3.2 Heights are worse than you think

There are at least three different things people mean by "height", and mixing them produces errors of tens of metres.

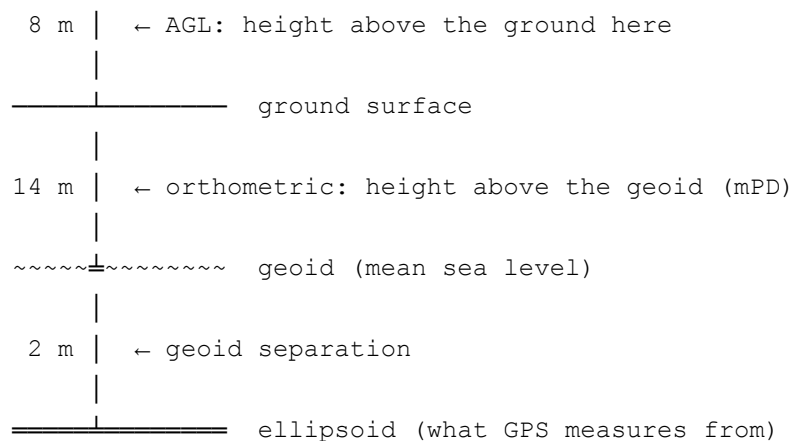
Ellipsoidal height. Height above a smooth mathematical ellipsoid approximating the Earth. This is what raw GNSS gives you. It has no physical meaning — a lake can have different ellipsoidal heights at each end.

Orthometric height. Height above the **geoid**, which is the surface of constant gravitational potential that mean sea level would follow. This is "height above sea level" as ordinarily understood, and it's what water responds to. In Hong Kong the local datum is **mPD** (metres above Principal Datum).

The difference between the two — the **geoid separation** — is not small. It's about +2 m in Hong Kong, around -30 m in parts of India, and over +60 m near Papua New Guinea. Converting requires a geoid model, not a constant.

Height above ground level (AGL). Height above the terrain directly beneath. This is what a drone's barometer or downward rangefinder reports, and what you mean when you say "the camera is mounted at 8 metres".

▲ camera
|



Rule: every stored height carries a label saying which of the three it is. A camera “at 8 m” is meaningless without it, and a camera stored with the wrong one ends up 14 m underground in the model.

3.3 Floating point will destroy your model if you let it

This is a specific, non-obvious trap that will bite in the 3D viewer.

Computer graphics runs on **32-bit floating point** numbers. A 32-bit float has 24 bits of significand, which means it can represent roughly 7 significant decimal digits — no matter how big the number is. The *absolute* precision therefore gets worse as numbers get larger.

Hong Kong Grid eastings are around 830,000 metres. In that range, consecutive representable 32-bit floats are about **6 centimetres apart**.

So if you load geometry into a WebGL viewer using raw projected coordinates, every vertex position is quantised to 6 cm *before anything is drawn*. Your GSD was 1 cm. You have thrown away all of it, plus more. The visible symptom is geometry that shimmers and jitters as the camera moves, and measurements that disagree with the database by a few centimetres for no apparent reason.

The fix — do this from day one: every site stores a **local origin**, a single anchor point in the projected CRS. All geometry is stored in the database in full projected coordinates, and rebased to that origin exactly once on load. The renderer and the coverage kernel work entirely in a local frame where coordinates run from about –200 to +200 metres. At that magnitude, float32 spacing is about 15 micrometres.

This gives us the three frames the system uses:

Frame	Units	Where used
Storage	Projected CRS, metres (e.g. EPSG:2326)	PostGIS, exports, site plans

Compute	Local ENU (East–North–Up) metres from the site origin	Coverage kernel, geometry maths
Display	Same local ENU, Y-up	The browser renderer

“ENU” is East–North–Up: a local Cartesian frame with X east, Y north, Z up. The renderer uses Y-up instead of Z-up purely because that’s the graphics convention; it’s a relabelling, not a different frame.

4. From a mess of points to a set of objects

We now have a metrically correct dense point cloud and a mesh. Neither is usable for computation. This section turns them into the proxy model from §1.

4.1 What we’re aiming for

Input: 200 million coloured points.

Output: something like this —

```

6 × light mast      cylinder, base (x,y), radius 0.28 m, height 15.2 m
4 × fence run      vertical polyline, height 2.4 m, thickness 0.12 m, porosity
1 × spectator stand box, 44 × 6 × 6 m
1 × pavilion        box, 18 × 8 × 4.5 m
2 × tree            box, 6.4 × 6.4 × 8 m, flagged seasonal
1 × terrain         height grid at 0.5 m spacing

```

Each with a confidence score, the points that support it, and an accept/reject state. About twenty objects instead of 200 million points.

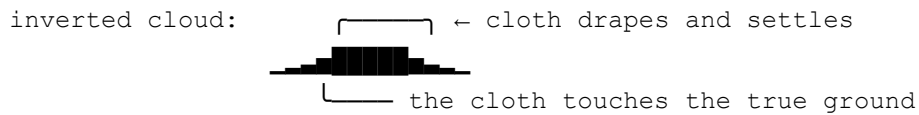
4.2 Step one: separate ground from everything else

The first useful cut is: which points are the ground, and which are things standing on it?

The best method for open sites is the **Cloth Simulation Filter**. The idea is delightfully physical:

1. Turn the point cloud upside down.
2. Drape a virtual sheet of cloth over it, from above, under gravity.
3. Let the cloth settle. It rests on the highest points of the inverted cloud — which are the *lowest* points of the real one.

- Points within a small threshold of the settled cloth are ground. Everything else is above-ground.



Cloth stiffness is the main tuning parameter: a stiff cloth won't follow real terrain undulations; a floppy one drapes into hollows it shouldn't. Sports grounds are nearly flat, so this is easy here.

Output: a DTM (the ground) and a residual cloud (everything else).

4.3 Step two: cluster the leftovers into objects

The above-ground points form obvious blobs — this mast, that fence run, that tree. Separating them is **clustering**.

Two standard approaches:

Euclidean clustering. Start with a point. Add every point within distance ϵ of it. Add every point within ϵ of those. Continue until nothing new is added; that's one cluster. Repeat from an unassigned point. Effectively connected components under a distance threshold. Simple and fast.

DBSCAN. Similar, but with a density requirement: a point only propagates a cluster if it has at least *minPts* neighbours within ϵ . This means sparse noise doesn't chain clusters together, and stray points get labelled as noise instead of forming spurious objects. Better for real data.

Before either, **voxel downsample** the cloud: divide space into a grid of 5 cm cubes and replace all points in each cube with their centroid. This cuts the point count by an order of magnitude with almost no information loss, and clustering cost scales badly with point count.

The parameter ϵ matters: too small and a single mast splits into segments; too large and a mast merges with the fence it stands next to. This is one of the places human review earns its keep.

4.4 Step three: fit shapes, using RANSAC

Now, for each cluster, decide what shape it is and fit that shape's parameters.

RANSAC — Random Sample Consensus — is the workhorse, and it's worth understanding because it appears throughout this pipeline.

The problem it solves: you have data that is mostly a clean shape plus some fraction of garbage. Ordinary least-squares fitting is destroyed by garbage, because a single far-off point can drag the fit arbitrarily. RANSAC ignores the garbage instead of averaging it in.

The algorithm:

1. Randomly pick the minimum number of points needed to define the shape. (Two points define a line; three define a plane.)
2. Fit the shape to exactly those points.
3. Count how many of *all* the points lie within a tolerance of that shape. Those are **inliers**.
4. Remember the fit with the most inliers.
5. Repeat a few hundred times.
6. Finally, refit properly using all the inliers of the best candidate.

It works because if you sample randomly enough times, eventually you'll pick a minimal set that happens to be all inliers, and that fit will attract a large consensus. Garbage points never form a large consensus with each other.

Now the shape signatures. Compute descriptors per cluster from the covariance of its point positions (the eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$ of the point distribution tell you whether it's spread out like a line, a plane, or a blob):

Class	How you recognise it	Fitted primitive
Light mast	Very tall, very thin in plan (< 0.4 m radius), points strongly aligned vertically, RANSAC cylinder fits with high inlier ratio	Cylinder: base point, radius, height, plus a bounding box for the lamp head
Fence / railing	Thin in one horizontal direction, long in the other, RANSAC vertical plane fits well, plan-view projection forms a line or polyline	Vertical extruded polyline: height, thickness
Building	Large volume, dominant near-horizontal plane above ground level	Oriented bounding box, or extruded footprint polygon
Goal / net frame	2–3 m tall, rectangular outline, low point density (open mesh)	Box
Tree / vegetation	Points spread in all three directions roughly equally, no good plane or cylinder fit	Box, flagged seasonal

Ground / court	Large near-horizontal plane	Already handled by the DTM
-----------------------	-----------------------------	----------------------------

Note the fence entry produces a *polyline*, not a single plane. Real perimeter fences turn corners. Fit a plane, project inliers to plan view, then fit a polyline with corner detection.

4.5 Step four: get semantics from the photographs

Geometry alone cannot tell a floodlight mast from a flagpole from a CCTV pole from a lightning conductor. They're all thin vertical cylinders. But they are visually completely distinct, and you have hundreds of photographs.

So: run a 2D segmentation model over the source images and lift the results into 3D.

2D segmentation means classifying every pixel in an image, producing a mask — “these pixels are a mast, those are grass”. Two practical options:

- **SAM 2** (Segment Anything) produces high-quality masks without knowing what anything *is* — class-agnostic. Pair it with a small classifier trained on cropped mask regions to assign a class.
- **A fine-tuned YOLO-seg** trained directly on your classes. Needs annotated drone imagery of your kind of site, but the annotation is ordinary 2D image labelling.

Lifting to 3D — back-projection voting. This is where the camera poses saved back in §2.4 finally pay off:

1. For each 3D point in the cloud, and each image that saw it:
2. Project the point into that image using the camera's known pose and intrinsics. That gives a pixel coordinate.
3. **Check visibility.** The point might be behind something from that viewpoint. Use the dense reconstruction's depth map: if the depth at that pixel is much less than the point's distance, the point is hidden — skip this image.
4. Read the segmentation label at that pixel. That's one vote.
5. After all images, the point's label is the most-voted class.
6. A cluster's label is the majority label of its points.

The reason to do it this way rather than train a 3D point-cloud network (PointNet++, KPConv, RandLA-Net) is training data. Annotating 3D point clouds is slow, specialist, and there's very little public data for this domain. Annotating 2D images is fast, cheap, and can be outsourced. You get 3D semantics from 2D labour.

4.6 Step five: the human review, which is the point of all this

Every candidate is presented to a reviewer as:

- The **fitted primitive**, drawn in 3D
- Its **class** and **confidence**
- Its **supporting points**, highlighted
- **Thumbnail crops** from the two or three source photographs that saw it best

The reviewer can:

- **Accept** — it goes into the occlusion model.
- **Reject** — with a typed reason (below).
- **Reclassify** — change the class. This must **trigger a refit**, not just relabel. A cluster called "fence" and a cluster called "wall" get different thicknesses, and therefore cast different shadows.
- **Adjust** — drag a mast's height, extend a fence run through a gap where the reconstruction failed. Manual override is not cheating; it's the fastest way to fix what the algorithm got wrong, and it's recorded as manual.

Rejection is typed, and the three types behave differently:

Type	Meaning	Effect
Noise	A reconstruction artefact. Not a real thing.	Discarded. Not an occluder.
Transient	Real on survey day, not permanent — a parked van, a skip, a match-day marquee.	Excluded from the occlusion model but kept in the record , so the map can be reproduced and so you know why the survey looked like that.
Seasonal	Real and permanent but variable — deciduous trees.	Included, but tagged so coverage can be computed with and without them. A survey flown in February and a coverage report used in July describe different sites.

Untyped rejection loses information you will want later. "Why is there no fence along the east side in the 2025 model?" is answerable if the rejection was typed and dated.

4.7 Why this step is the hinge of the whole system

Consider a coverage map with a mysterious dark patch.

Without the review step, the answer is “some triangles in the mesh block the ray”. Nobody can check that, argue with it, or fix it.

With the review step, the answer is “structure S-041, *Tree 2*, accepted on 2026-08-14 by J. Wong, class *vegetation*, flagged seasonal”. You can look at it, disagree, reclassify it, and recompute in 25 milliseconds.

That difference — between a system whose outputs are opaque and one whose outputs are traceable — is what makes it usable for something as consequential as specifying a CCTV installation.

5. Measuring things

5.1 Snapping, or why raw clicking doesn't work

Click a point cloud twice to measure a distance and you get noise. Your two clicks land on two individual points that each sit a few centimetres off the true surface, in depth as well as laterally. Measure again and you get a different answer.

Snapping fixes this by attaching the click to something meaningful rather than to whatever point happened to be under the cursor. Priority order:

1. **Fitted primitive features** — a mast's axis, a mast's top, a fence line, a building edge or corner. Best, because these are already least-squares fits over thousands of points.
2. **The DTM surface** — for ground positions.
3. **A locally fitted plane** — take the points within a small radius of the cursor, fit a plane, snap to it. Averages out noise.
4. **Nearest raw point** — last resort.

This is the difference between a tool a surveyor will use and one they won't.

5.2 What you can measure

Measurement	Notes
3D distance	Straight-line, point to point

Horizontal distance	Plan distance, ignoring height
Vertical difference	Height between two points
Height above ground	Perpendicular to the <i>locally</i> fitted ground, not to an assumed flat plane
Polyline length	Following a path
Planar area	Pitch area, court area, footprints
Volume	Between a surface and a reference plane — for stockpiles or excavation
Slope / gradient	Drainage falls, ramp compliance
Clearance	Minimum distance between two objects

5.3 Uncertainty, and not lying with decimal places

Every measurement carries an uncertainty. It comes from:

- **GSD** — you cannot resolve anything smaller than a pixel on the ground.
- **Reprojection RMSE** — how well the bundle adjustment converged.
- **Check-point residuals** — the measured discrepancy against surveyed truth.
- **Snapping mode** — a snap to a fitted cylinder axis is far more certain than a snap to a raw point.

These combine roughly in quadrature (square root of the sum of squares, since the errors are largely independent).

Report the result as $47.82 \text{ m} \pm 0.03$, never as 47.8213 m .

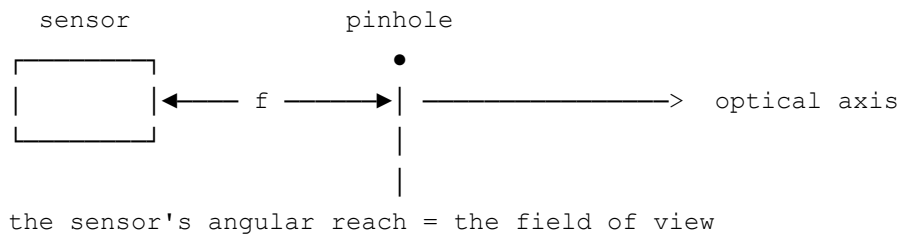
Displaying four decimal places on a quantity known to $\pm 3 \text{ cm}$ is a lie told with precision. It will be believed, quoted in a document, and eventually relied upon. The tolerance must travel with the number into every export.

6. Cameras and what they can see

Now the part the whole system exists to support.

6.1 The pinhole model

For planning purposes, a camera is a pinhole: a single point through which all light rays pass, and a rectangular sensor behind it at distance f — the **focal length**.



Real lenses distort, but distortion changes *where within the frame* something appears, not *whether* it's in frame or *how big* it is, to first order. For coverage planning the pinhole is fine. (You would need the distortion model to overlay a simulated image on a real one, which is a later feature.)

6.2 Field of view

A larger sensor sees more; a longer lens sees less. From simple trigonometry:

$$\begin{aligned}\text{HFOV} &= 2 \cdot \arctan(\text{sensor_width} / (2f)) \\ \text{VFOV} &= 2 \cdot \arctan(\text{sensor_height} / (2f))\end{aligned}$$

Worked example — a common CCTV configuration, a 1/2.8" sensor (5.37×4.04 mm) with an 8 mm lens:

$$\begin{aligned}\text{HFOV} &= 2 \cdot \arctan(5.37 / 16) = 2 \cdot 18.55^\circ = 37.1^\circ \\ \text{VFOV} &= 2 \cdot \arctan(4.04 / 16) = 2 \cdot 14.13^\circ = 28.3^\circ\end{aligned}$$

That "1/2.8 inch" naming is a historical fiction inherited from vacuum tube cameras and bears no useful relation to any actual dimension. Always work from the millimetre dimensions.

Shorter lens $\rightarrow$ wider view $\rightarrow$ less detail per object. Longer lens $\rightarrow$ narrower view $\rightarrow$ more detail. This is the central trade-off of every CCTV design, and it is exactly what the tool exists to make visible.

6.3 Focal length in pixels, and the key formula

Here is the formula that everything about image quality reduces to.

Consider a flat plane at distance d from the camera, perpendicular to the optical axis. How much of that plane fits in the frame, vertically?

$$\text{vertical extent visible} = 2 \cdot d \cdot \tan(\text{VFOV}/2) \quad \text{metres}$$

The sensor covers that with `image_height` pixels. So:

$$\text{pixels per metre} = \text{image_height} / (2 \cdot d \cdot \tan(\text{VFOV}/2))$$

Define **focal length in pixels**:

$$f_px = \text{image_height} / (2 \cdot \tan(\text{VFOV}/2))$$

which, equivalently and more directly, is:

$$f_px = \text{focal_length_mm} \times \text{image_height_px} / \text{sensor_height_mm}$$

and the whole thing collapses to:

$$\text{px/m} = f_px / d$$

Pixel density on target falls off inversely with distance. That's it.

Worked example — 8 mm lens, 1/2.8" sensor, 4K (3840 × 2160):

$$f_px = 8 \times 2160 / 4.04 = 4277 \text{ pixels}$$

At 20 m: $4277 / 20 = \mathbf{214 \text{ px/m}}$. At 50 m: $4277 / 50 = \mathbf{86 \text{ px/m}}$. At 100 m: $4277 / 100 = \mathbf{43 \text{ px/m}}$.

6.4 DORI: what those numbers mean

Coverage is not binary. A cell is not "covered" or "not covered"; it is covered *to a standard*. The standard is **DORI**, from IEC EN 62676-4, and it defines four thresholds in pixels per metre on the target:

Tier	px/m	What it supports
Detect	25	You can tell that a person is there
Observe	62	You can tell what they are doing
Recognise	125	You can tell whether it's someone you know
Identify	250	Sufficient for evidential identification of a stranger

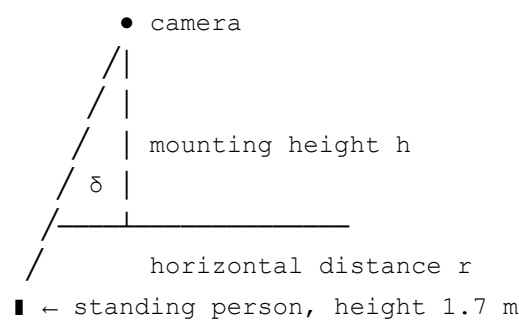
So the 8 mm 4K camera above gives, head-on:

```
Identify    to 4277 / 250 = 17 m
Recognise   to 4277 / 125 = 34 m
Observe     to 4277 / 62  = 69 m
Detect      to 4277 / 25  = 171 m
```

That is the honest answer to “how far can this camera see”, and it is four different answers depending on what you need it to do. A tool that reports a single coverage footprint is answering the wrong question.

6.5 Foreshortening, and why mounting higher backfires

The formula above assumes a target perpendicular to the line of sight. But you’re looking down at people, and a person is a **vertical** target.



A vertical target viewed from a depression angle δ occupies less of the frame than it would head-on, by a factor of $\cos \delta$. In the limit — camera directly overhead — you see the top of someone’s head, and their height occupies zero pixels.

$$\text{px/m}_{\text{effective}} = (f_{\text{px}} / d) \cdot \cos \delta$$

where $d = \sqrt{h^2 + r^2}$ is the slant range, and $\cos \delta = r / d$. So the whole thing is:

$$\text{px/m}_{\text{effective}} = f_{\text{px}} \cdot r / d^2 = f_{\text{px}} \cdot r / (h^2 + r^2)$$

Now the counterintuitive bit. Take the 8 mm 4K camera, $f_{\text{px}} = 4277$, and a person 30 m away horizontally:

Mounting height	Slant range	$\cos \delta$	Naive px/m	True px/m	Tier
6 m	30.6 m	0.981	140	137	Recognise
14 m	33.1 m	0.906	129	117	Observe only

25 m	39.1 m	0.768	109	84	Observe only
------	--------	-------	-----	-----------	--------------

Raising the camera from 6 m to 25 m cuts usable image quality on a standing person by nearly 40%, at the same horizontal distance. Height buys you sightlines over obstacles; it costs you pixels on people. Both effects are real and they oppose each other. The tool's job is to let you find the balance rather than guess.

A coverage tool that reports **flat ground footprint area** will happily recommend a 25 m mast mounting and show a big satisfying green blob, for an installation that cannot recognise anybody. Hence the design rule:

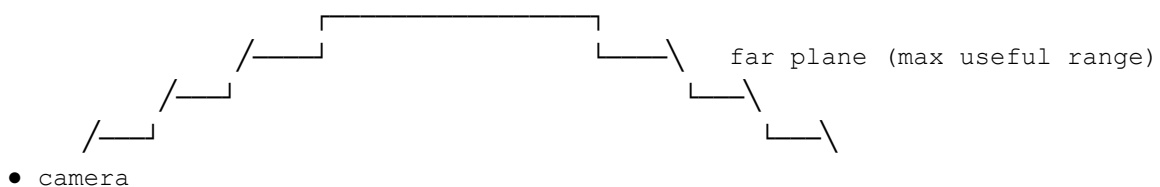
Evaluate coverage against vertical targets at 1.6 m above ground — chest height on a standing adult — not against the ground plane.

In the prototype, turning foreshortening off inflates “Recognise or better” from 5.2% to 7.6% on the identical scene. That gap is entirely fictional coverage.

There's a second angular effect worth knowing about: a target viewed *obliquely in plan* (from the side, rather than face-on) is also foreshortened, which matters for facial identification specifically — faces need the camera within roughly 30° of straight-on. That's a refinement for a later phase; the depression-angle term is the one that dominates.

6.6 The view frustum

The volume a camera can see is a **frustum** — a pyramid with its apex at the lens, truncated by a near and a far plane.



Testing whether a point is inside is straightforward once you express it in the camera's own coordinate frame. Build three perpendicular unit vectors:

- **forward** — the direction the camera points, from its pan and tilt
- **right** — perpendicular to forward, horizontal
- **up** — perpendicular to both

Then for a world point P , with $\mathbf{v} = P - \text{camera_position}$:

$$z = \mathbf{v} \cdot \text{forward} \quad (\text{distance along the optical axis})$$

```

x = v · right
y = v · up

in frustum ⇔ z > 0
               and |x| ≤ z · tan(HFOV/2)
               and |y| ≤ z · tan(VFOV/2)
               and near ≤ |v| ≤ far

```

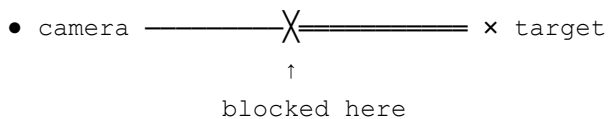
Four dot products and three comparisons. This is the innermost loop of the coverage computation and it needs to be this cheap.

7. Occlusion: what is hidden behind what

Being inside the frustum is necessary but not sufficient. Something may be in the way.

7.1 Ray casting

The test is: draw a line segment from the camera to the target point. Does it hit anything first?



Mathematically, parameterise the segment as $\text{origin} + t \cdot \text{direction}$, with t running from 0 at the camera to 1 at the target. For each occluder, solve for the t at which the ray enters it. If any occluder yields a t strictly between 0 and 1, the target is hidden.

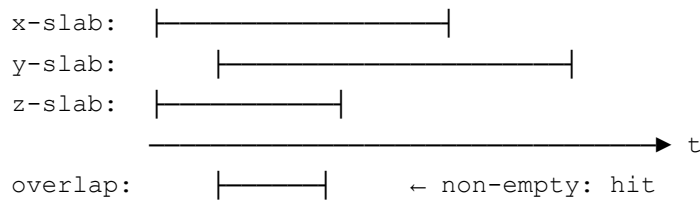
The efficiency of this depends entirely on what the occluders are. Against 40 million triangles it is slow and needs elaborate acceleration structures. Against 20 analytic primitives it is trivially fast — which is the payoff from §4.

7.2 Ray versus box: the slab method

Boxes are the workhorse — buildings, fence runs, tents.

An axis-aligned box is the intersection of three **slabs**: the region between two parallel planes, one pair per axis.

For each axis independently, work out the interval of t during which the ray is between that axis's two planes. That's two divisions. Then intersect the three intervals. If the result is non-empty, the ray passes through the box, and the interval's start is the entry point.



A **rotated** box (a tent at an angle) is handled by transforming the ray into the box's own coordinate frame — rotate the origin and direction by the negative of the box's yaw — and then running the ordinary axis-aligned test. This is much cheaper than testing against six arbitrary planes.

7.3 Ray versus cylinder

Light masts. For a cylinder with a vertical axis, drop the height component and solve a 2D circle intersection: substituting the ray equation into the circle equation gives an ordinary quadratic in t . Solve it, then check that the y coordinate at the solution lies between the cylinder's base and top.

7.4 Broad phase: don't test what can't possibly hit

Before running the exact intersection test, two cheap rejections:

Vertical extent. If the entire segment lies above the occluder's top or below its bottom, skip it. Cameras are high, targets are at 1.6 m, and a 2.4 m fence is below most of that path — this rejects a lot of tests.

Plan-view distance. Compute the 2D distance from the occluder's centre to the segment, and compare it against the occluder's bounding radius. If it's greater, no intersection is possible. This is one clamped dot product and a couple of multiplies.

These take the cost from "every ray against every occluder" to "every ray against the two or three occluders anywhere near it".

7.5 The grid, and the aggregation

Sample the area of interest on a regular grid — 0.25 m for a final report, 1.0 m for a quick look. For each cell, evaluate at the target height (1.6 m), and for each camera:

1. Range gate
2. Frustum test
3. Occlusion test
4.
$$p_x/m = (f_{px} / d) \cdot \cos \delta$$

Then aggregate across cameras, keeping two numbers per cell:

- **Best pixel density** — the maximum across all cameras. This is what the operator actually gets, since they'd use the best view available.
- **Camera count** — how many cameras can see this cell at all.

The second number is not optional. Redundancy matters for two practical reasons: tracking someone across a site requires handover between overlapping cameras, and a single-camera area becomes a blind spot the moment that camera fails or its lens gets dirty. A coverage report that only shows the maximum is incomplete.

Derived outputs: the percentage of the site meeting each DORI tier, the percentage seen by two or more cameras, and blind-spot polygons — contours traced around the zero-coverage regions, extracted with marching squares.

7.6 Two implementations, deliberately

Interactive (GPU). Render the proxy model into a depth buffer from the camera's own viewpoint — this is a **shadow map**, the standard real-time graphics technique for shadows. Then for each grid cell, project it into that camera and compare its depth against the stored depth. Nearer means visible; further means something's in front. This runs in real time, so you can drag a camera and watch the map change continuously.

Authoritative (CPU). Ray-cast against the primitives, exactly as described above. Deterministic, no precision artefacts, slower. Used for anything that ends up in a document.

Why both? Because depth buffers have finite precision, and at grazing angles — a camera looking almost along a wall — they produce artefacts (surfaces incorrectly shadowing themselves). Those are acceptable in a viewport where you're dragging things around, and unacceptable in a tender document.

The two implementations get cross-checked in the test suite. Divergence beyond a tolerance is a bug in one of them, and it is almost always the shadow map.

7.7 The self-occlusion trap

Every pole-mounted camera, without exception, will find its own pole between itself and half the site — because the camera sits *on* the cylinder that the occlusion test is checking against.

The fix has two parts:

1. **Model the bracket.** Cameras aren't mounted at the pole's centreline; they're on an arm 0.5–1 m out. Store that offset and start the ray from the actual lens position.

2. **Exclude the parent structure** from that camera's occluder set.

Do both. Every symptom of this bug looks like something else — “the west half of the site is blind for no reason” — and it takes a surprisingly long time to find.

7.8 Fence porosity

Chain-link and welded mesh fencing is geometrically a solid vertical plane and optically nearly transparent. You can see straight through a chain-link fence at 30 m, and barely at all at a grazing angle where you're looking through many layers of wire.

Modelling it as a solid occluder puts large false shadows on the map. Modelling it as absent overstates coverage.

The correct treatment is an **attenuation factor** per structure: a fence with porosity 0.85 doesn't block the ray, it reduces the effective pixel density behind it (image contrast through mesh is degraded, so the usable px/m drops). The attenuation should ideally scale with the incidence angle, since a grazing view passes through more wire.

The prototype models fences as solid, which is the conservative choice. The full system should not.

8. Tents, scenarios, and choosing where cameras go

8.1 Tents

A tent is a box: position, footprint, height, rotation. In the system it plays two roles:

- **An occluder**, entering the ray tests exactly like a building.
- **An area of interest** in its own right — the aisles between tents at an event are usually the part that most needs watching.

8.2 Scenarios, not just moveable boxes

The valuable question isn't “what's the coverage with these tents here”. It's “**what changes**”. So tents are grouped into named **scenarios** — “match day”, “carnival layout”, “typhoon shelter configuration” — attached to a base survey, and coverage is compared across them.

The headline output is a **delta**:

Erecting 12 tents takes blind area from 8.3% to 21.2% and drops 1,400 m² below its previous coverage entirely. That is the area a temporary camera would have to cover.

That sentence is the actual deliverable — it converts a visualisation into a procurement decision. It's why scenarios are a first-class concept rather than an incidental consequence of letting users drag boxes around.

Seasonal structures work the same way: compute with and without the seasonal set, and report the winter/summer difference.

8.3 Automatic camera placement

Once the coverage kernel exists, this comes almost free, and it's genuinely valuable.

Candidate mounting points come from the accepted structures. Mast tops, fence posts, building corners and roof edges are exactly the places hardware can physically go. §4 already found and classified them.

Discretise each candidate into a set of options — a few pan angles, a few tilts, a few lens choices. For each option, precompute which grid cells it covers at each DORI tier, stored as a bitset.

This is now the classic **maximum coverage problem**: choose k sets from a collection to cover as many elements as possible. It's NP-hard, so exact optimisation is impractical at realistic sizes. But the **greedy** algorithm — repeatedly pick whichever remaining option adds the most *new* coverage — has a proven guarantee: it always achieves at least $1 - 1/e \approx 63\%$ of the optimal coverage, and in practice usually much closer to optimal than that.

Greedy is also explainable, which matters more than it sounds. "We picked this position because it adds 4,200 m² that nothing else covers" is a sentence you can put in front of a client. The output of an integer program is not.

Constrain by cable routing distance, mounting height limits, and structural load.

9. How the software is put together

9.1 Package structure

```
apps/  
  web/      React + TypeScript, three.js for 3D, streams tiled geometry  
  api/      FastAPI — projects, jobs, structures, cameras, scenarios, exports  
  worker/   Long-running photogrammetry and segmentation jobs  
  cli/      Headless: point it at a folder of images, get a coverage report  
  
packages/  
  contracts/ The data shapes, defined once, shared by everything
```

geo/	CRS conversion, local-origin rebasing, height datums, GSD/FOV maths
recon/	Drives OpenDroneMap and COLMAP; validates their outputs
segment/	Ground filtering, clustering, primitive fitting, 2D→3D lift
coverage/	The kernel

The **dependency direction only points one way**: apps depend on packages, packages depend on contracts, contracts depend on nothing. No package ever imports an app. This sounds like bureaucracy until the first time you need to run the coverage kernel from a script without starting a web server.

contracts/ is the middle of the hourglass. Define the data shapes once in Python (Pydantic), generate the TypeScript types from them. The frontend and backend then cannot disagree about what a camera is, because there is only one definition. Be strict about what you emit and lenient about what you accept — a client one version behind should ignore fields it doesn't recognise rather than crash.

coverage/ has no I/O and no framework dependency. It is pure functions: geometry in, numbers out. This matters because it's the part whose correctness is hardest to eyeball — a wrong coverage map still looks like a coverage map — and easiest to test exhaustively. It also needs to run identically in three places: the worker, the CLI, and (compiled to WebAssembly) the browser.

9.2 The database

PostgreSQL with PostGIS and TimescaleDB.

PostGIS adds real geometry types to Postgres — points, lines, polygons, with proper coordinate reference system handling — plus spatial indexing and spatial queries ("which structures are within 50 m of this camera"). It is not optional here. Structures, tents, cameras, areas of interest and blind-spot polygons are all geometry, and doing spatial work in the database is faster and more correct than pulling everything into Python and looping.

Large artefacts go to object storage, not the database. Point clouds, meshes, orthophotos and tile sets are hundreds of megabytes to tens of gigabytes. The database stores a URI and a checksum.

9.3 Streaming: you cannot send a point cloud to a browser

A dense cloud of a sports ground is 200–400 million points. Even compressed that's gigabytes, and no browser will hold it.

The answer is **level-of-detail tiling**, standardised as **3D Tiles**. Space is subdivided hierarchically — an octree. The root tile is a coarse representation of the whole site; children are progressively finer representations of smaller regions. The viewer computes, for each

tile, its **screen-space error**: how wrong it would look at the current viewing distance. Tiles above the error threshold get refined by loading their children; distant ones stay coarse.

The result is that you download a few megabytes for an overview, and detail streams in only where you're actually looking. Tools: `py3dtiles`, or Entwine/EPT for the underlying indexing. Meshes go as Draco-compressed glTF.

The proxy model, by contrast, is tiny. A few hundred primitives is a few kilobytes of JSON. It loads instantly, and it's what all the interaction happens against. The tiled point cloud is a backdrop for context and for measuring against; the proxy is the model the system actually computes on. Keeping them separate is what makes the tool feel fast.

9.4 Jobs

Reconstruction takes 20 minutes to 6 hours depending on image count, image size, and scene difficulty. That's not a request-response operation.

Every long operation is a **job**:

- Persisted state, so a worker restart doesn't lose it
- Progress streamed to the browser over WebSocket
- Resumable, or at least restartable from the last completed stage
- Producing **versioned artefacts**

That last point is important and easy to get wrong. **A reconstruction is a version of a site, not a mutation of it.** Re-flying a site next year must not invalidate last year's coverage report — you need to be able to open a report from 2026 and see the model it was computed against. So surveys are immutable once complete, and everything downstream references a specific survey ID.

Similarly, `coverage_run` records the **kernel version** used. Coverage numbers end up in tender documents. When you fix a bug in the kernel, you need to know which results predate the fix.

9.5 Compute placement

Dense matching in ODM and 2D segmentation both want a GPU. The API and database don't. Split them: a GPU worker box, and ordinary infrastructure for everything else.

9.6 The data model, annotated

```
site      id, name, crs, local_origin(x,y,z)
          └ the local_origin is the float32 fix from §3.3
```



```

survey      id, site_id, flown_at, platform, georef_method(rtk|gcp|scale|none),
            image_count, gsd_m, check_rmse_h, check_rmse_v, status
            └ georef_method drives whether dimensioning is enabled at all
            └ check_rmse_* are the honest accuracy numbers from $2.8

artefact     id, survey_id, kind(pointcloud|dsm|dtm|ortho|mesh|tiles), uri, sha25
            └ checksums, because these files get moved and re-uploaded

structure    id, survey_id, class, confidence, state(pending|accepted|rejected|
            seasonal), reject_reason, primitive(jsonb), footprint(geometry),
            porosity, reviewed_by, reviewed_at
            └ this table IS the occlusion model
            └ reviewed_by/at is the audit trail that makes maps defensible

mount_point  id, structure_id, position(geometryz), max_load_kg
            └ extracted automatically; feeds the placement optimiser

camera       id, scenario_id, mount_point_id, position(geometryz), pan, tilt, rol
            bracket_offset_m, sensor_w_mm, sensor_h_mm, focal_mm, res_x, res_y,
            near_m, far_m, model_name
            └ mount_point_id is what excludes the parent from self-occlusion

scenario     id, site_id, name, base_survey_id, seasonal_included
tent         id, scenario_id, footprint(geometry), height_m, yaw_deg

coverage_run id, scenario_id, eval_height_m, grid_m, method(bvh|shadowmap),
            computed_at, kernel_version, stats(jsonb), grid_uri

measurement  id, site_id, kind, geometry, value, uncertainty, created_by
            └ uncertainty is a column, not a rendering detail

```

10. How you know any of it is correct

Nothing in this system is trustworthy without a number attached to it.

Layer	Test	Reported as
Reconstruction	Check points held out of the bundle adjustment	RMSE horizontal and vertical, per survey
Scale	Reconstructed pitch/court markings vs. regulation dimensions	Percentage scale error

Structure extraction	Manual review of a held-out set; plus the production accept/reject rate	Precision and recall per class
Dimensioning	Tape or total station on ten features	Mean absolute error
Coverage kernel	Shadow-map vs. ray-cast agreement; analytic cases with closed-form answers	Max divergence, percentage of cells differing
Coverage in reality	Photograph from an actually-installed camera vs. the simulation	Per-camera comparison

The analytic cases deserve a note, because they're how you catch the subtle bugs. Set up a single camera pointing straight down from a known height over flat ground with no occluders. The footprint must be exactly $2h \cdot \tan(HFOV/2) \times 2h \cdot \tan(VFOV/2)$ and the peak pixel density must be exactly f_{px} / h . If the code disagrees with arithmetic you can do on paper, the code is wrong. (The prototype passes this: measured 63.0 m² against 60.3 m² expected, the difference being grid quantisation over a small footprint, and 213.7 px/m against 213.9 predicted.)

Another good check: **grid invariance**. The reported percentages should barely change between 1.0 m, 0.5 m and 0.25 m grid spacing. If they drift, something is resolution-dependent that shouldn't be.

The last row is the one that gets skipped, and it is the one that matters. A coverage tool that has never been checked against a real installed camera is a plausible-looking picture generator. Install one camera early, photograph a test target at measured distances, count the pixels on it, and compare against what the model predicted. Budget for this.

11. Build order

Phase 0 — the coverage kernel on synthetic geometry. No drone, no photogrammetry, no reconstruction. Hand-authored site, cameras, tents, full DORI computation with occlusion and redundancy, and the visualisation. This proves the hardest and most novel part first, and it is immediately useful on any site where a CAD plan already exists. *This is what `groma-coverage-prototype.html` does.*

Phase 1 — import and measure. Ingest existing LAS/OBJ/GeoTIFF reconstructions from any source. 3D Tiles conversion, streaming viewer, snapping, dimensioning with uncertainty. Now useful to anyone who already has survey data.

Phase 2 — reconstruction in-house. ODM orchestration, the job queue, GCP and RTK

handling, accuracy reporting, the pitch-markings scale check. This is the long pole: expect it to take longer than everything before it combined.

Phase 3 — structure extraction and review. Cloth simulation filter, clustering, primitive fitting, 2D→3D mask lift, and the accept/reject review interface. Coverage switches from a hand-authored proxy to a derived one, and the system becomes end-to-end.

Phase 4 — planning and reporting. Scenarios, coverage deltas, greedy placement optimisation, and PDF export with blind-spot plans and per-camera schedules.

Phase 0 gives something demonstrable in a week or two. That ordering is deliberate: the riskiest and most differentiating component is built first, when there is still time to discover it's harder than expected.

12. Everything that will go wrong

1. **Scale-free SfM.** Without RTK or GCPs the model is dimensionally wrong and looks perfect. *Refuse to enable dimensioning on an unscaled reconstruction.* Don't warn — warnings get dismissed.
2. **Float32 at projected coordinates.** Rebase to a local origin, or watch geometry quantise to 6 cm before it reaches the screen. The symptom is jitter; the cause is not obvious.
3. **Thin structures reconstruct worst and matter most.** Masts and chain-link fences are simultaneously where photogrammetry is weakest and where camera planning is most sensitive. This is why oblique orbits and primitive fitting are requirements, not refinements.
4. **Chain-link fencing.** Geometrically solid, optically nearly transparent. Model porosity as an attenuation factor or your maps will show shadows that don't exist.
5. **Self-occlusion by the mount.** Every pole-mounted camera, silently, until fixed.
6. **Height datum confusion.** Ellipsoidal, orthometric and AGL mixed in one model puts cameras tens of metres underground.
7. **Depth-buffer precision.** The shadow-map coverage will disagree with the ray-cast coverage at grazing angles. Decide which is authoritative before someone notices the discrepancy in a meeting.
8. **Seasonal vegetation.** A survey flown in February and a coverage report used in July describe different sites. Tag seasonal structures and compute both.

9. **Reconstruction is slow and highly variable.** The same hardware takes 20 minutes or 6 hours depending on the survey. Design for asynchronous jobs from day one; retrofitting them later is painful.
 10. **Video ingest becomes the default.** If you offer it, it will be used, because it's easier to fly. Make the quality penalty visible at ingest time — rejection rates, sharpness histograms — not buried in the accuracy report afterwards.
 11. **Flat-footprint coverage reporting.** The single easiest way to produce a confident, beautiful, wrong answer. Always evaluate against vertical targets.
 12. **Over-precise measurements.** 47.8213 m on a quantity known to $\pm 3\text{ cm}$ will be believed and quoted. Carry the tolerance everywhere.
-

13. Glossary

AGL — Above Ground Level. Height measured from the terrain directly below.

Bundle adjustment — Simultaneous refinement of all camera poses, camera intrinsics, and 3D point positions to minimise total reprojection error.

Check point — A surveyed point deliberately excluded from the bundle adjustment, used afterwards to measure the model's true accuracy.

Cloth Simulation Filter (CSF) — Ground/non-ground separation by draping a virtual cloth over an inverted point cloud.

DBSCAN — Density-Based Spatial Clustering. Groups points that are densely packed together and labels sparse points as noise.

Descriptor — A numeric summary of the appearance around an image feature, constructed to remain similar under changes in viewpoint and lighting.

DORI — Detect / Observe / Recognise / Identify. The IEC EN 62676-4 standard's four image-quality tiers, defined in pixels per metre on the target: 25 / 62 / 125 / 250.

DSM — Digital Surface Model. Height grid of the top of everything.

DTM — Digital Terrain Model. Height grid of the bare ground.

Ellipsoidal height — Height above the reference ellipsoid. What raw GNSS gives.

ENU — East–North–Up. A local Cartesian coordinate frame anchored at a chosen point.

EPSG code — A number identifying a specific coordinate reference system in a public

registry. E.g. EPSG:2326 is the Hong Kong 1980 Grid.

Extrinsics — A camera's position and orientation in the world.

f_px — Focal length expressed in pixels: $\text{focal_mm} \times \text{image_height_px} / \text{sensor_height_mm}$. Pixel density on a target at distance d is $\text{f_px} / d$.

Feature — A small, distinctive, repeatably-detectable patch in an image.

Frustum — The truncated pyramid of space a camera can see.

GCP — Ground Control Point. A surveyed physical marker used to georeference a reconstruction.

Geoid — The equipotential gravitational surface that mean sea level approximates. The reference for orthometric heights.

GNSS — Global Navigation Satellite System. The generic term for GPS, Galileo, GLONASS, BeiDou collectively.

GSD — Ground Sample Distance. How many metres of ground one image pixel covers. Sets the accuracy ceiling for everything downstream.

Intrinsics — A camera's internal geometry: focal length in pixels, optical centre, lens distortion.

LAS / LAZ — The standard point cloud file formats. LAZ is the compressed form.

Maximum coverage problem — Choose k sets to cover as many elements as possible. NP-hard; the greedy algorithm guarantees at least $1 - 1/e \approx 63\%$ of optimal.

MVS — Multi-View Stereo. Computing dense per-pixel depth from images whose poses are already known.

Nadir — Straight down.

Occlusion — One object blocking the view of another.

Oblique — Camera angled between horizontal and straight down, typically 45°.

Orthometric height — Height above the geoid. "Height above sea level."

Orthophoto — An aerial image reprojected so every pixel is viewed straight down, removing perspective so distances can be measured directly.

Parallax — The apparent shift in an object's position when viewed from two different places. Larger for nearer objects; the basis of all depth from stereo.

Pinhole model — The idealisation of a camera as a single point through which all light rays

pass onto a flat sensor.

Point cloud — An unstructured set of 3D points, usually with colour.

Porosity — How much a nominally solid structure (mesh fence) actually lets you see through it.

Proxy model — The small set of approved primitives that stands in for the raw reconstruction in all computation. The heart of this system.

RANSAC — Random Sample Consensus. Fitting a model to data containing outliers by repeatedly fitting to random minimal subsets and keeping whichever fit gathers the most inliers.

Reprojection error — The pixel distance between where a reconstructed 3D point projects into an image and where its feature was actually detected. The fundamental quality metric.

Rolling shutter — Sensor readout row by row rather than all at once, causing geometric skew when the camera moves.

RTK / PPK — Real-Time / Post-Processed Kinematic GNSS. Centimetre-accurate positioning using a second receiver at a known point to cancel shared errors.

Screen-space error — In level-of-detail streaming, how many pixels wrong a coarse tile would look at the current viewing distance. Drives whether to load finer detail.

Segmentation — Assigning a class label to every pixel (2D) or every point (3D).

SfM — Structure from Motion. Recovering both camera poses and 3D scene structure from a set of overlapping photographs.

Shadow map — A depth image rendered from a light's (or camera's) viewpoint, used to test visibility quickly on a GPU.

Slab method — Ray-box intersection by intersecting the t -intervals during which the ray lies between each axis's pair of parallel planes.

Sparse cloud — The relatively few 3D points produced directly by SfM feature triangulation, before dense matching.

Similarity transform — Rotation, translation and uniform scaling: the seven degrees of freedom that raw SfM cannot determine.

3D Tiles — An open standard for streaming large 3D geospatial datasets with level-of-detail.

Voxel downsampling — Reducing point count by replacing all points in each small cube of space with a single representative point.

